What shipped in ptyd, Oct 27 to Nov 3

2018kguo/ptyd · 2018kguo · Oct 27 – Nov 3, 2026

ptyd is a terminal workspace for coding agents: you group terminals by task, run Codex, Claude Code and shells inside them, and come back later to watch or take over while they keep running.

Restarting ptyd brought terminals back subtly wrong, agent sessions from other tabs were invisible to it, and scrolling an attached agent lagged a frame behind a direct terminal. Over 27 PRs, restores became exact, terminals moved into persistent groups with a searchable grid of live previews, and scrolling paints in 0.15 ms instead of 16.7 ms.

Terminals live in named groups that keep running after you close the window

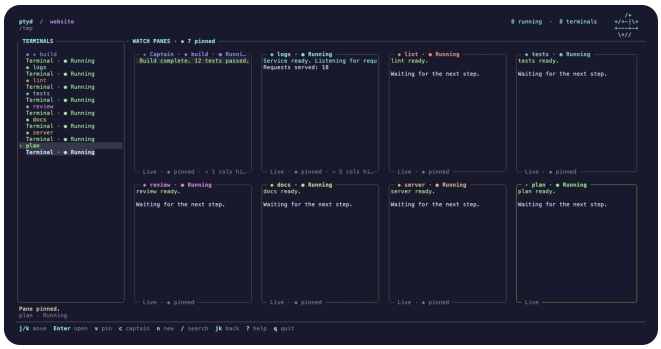
You can put each task in its own group, leave, and come back to every terminal where you left it.



From the README demo: a group's terminals on the left and a read-only live preview of the selected one.

Watch 8 terminals at once and search them by what is on screen

Finding the agent that printed an error no longer means opening tabs one by one.



From the README demo: seven pinned panes and the selected terminal in one group.

TL;DR

- New: named groups of terminals that outlive the window, with a live preview of each one.
- New: a grid of up to 8 live panes, and `/` to search every screen and its scrollback.
- Better: restarts restore terminals exactly, and scrolling an agent paints in 0.15 ms instead of 16.7 ms.

Meet the new pieces

PTY	A pseudo-terminal: the pipe a program like a shell or Codex reads keys from and writes its screen to. ptyd owns these, which is why your sessions survive closing the window.
checkpoint	A snapshot of a terminal that ptyd saves to disk so it can bring the terminal back after a restart. What it holds decides whether a restored terminal looks exactly like the one you left.
grapheme	What a reader sees as one character, such as a flag emoji, even when it is several code points. Agent output is full of them, so how fast ptyd splits them sets how fast it keeps up.

New features

Persistent terminal groups replace the flat picker, and the dock lists every working key Open a named group of terminals with `ptyd new`, `ptyd ls` or `ptyd attach`, preview and take over any of them, and press quick `jk` to go back from any level.

Up to 8 live terminal previews in one grid, searchable by screen text and scrollback Watch up to seven pinned panes plus the selected terminal, and press `/` to find a session by what is on its screen or in its scrollback.

Codex and Claude sessions from other tabs open in ptyd; a live check finds 5 of 5 Pick a running Codex or Claude conversation in ptyd and press Enter. Detaching returns to the list while the native worker keeps running.

Fixes

After ptyd restarted, your terminals came back subtly wrong: saved cursor positions gone, a full-screen app's second screen gone, and keys caught mid-sequence lost. Checkpoints store the emulator's exact snapshot, old version 1 files still load, and malformed history is rejected before it replaces the terminal.

Previews of Codex and Claude cut off their footers and the bottom of the input box, and pane colors bled into the gaps between panes. Previews keep the last painted row in view, and every cell outside a pane border matches the frame drawn with no previews, for one through eight panes.

Improvements

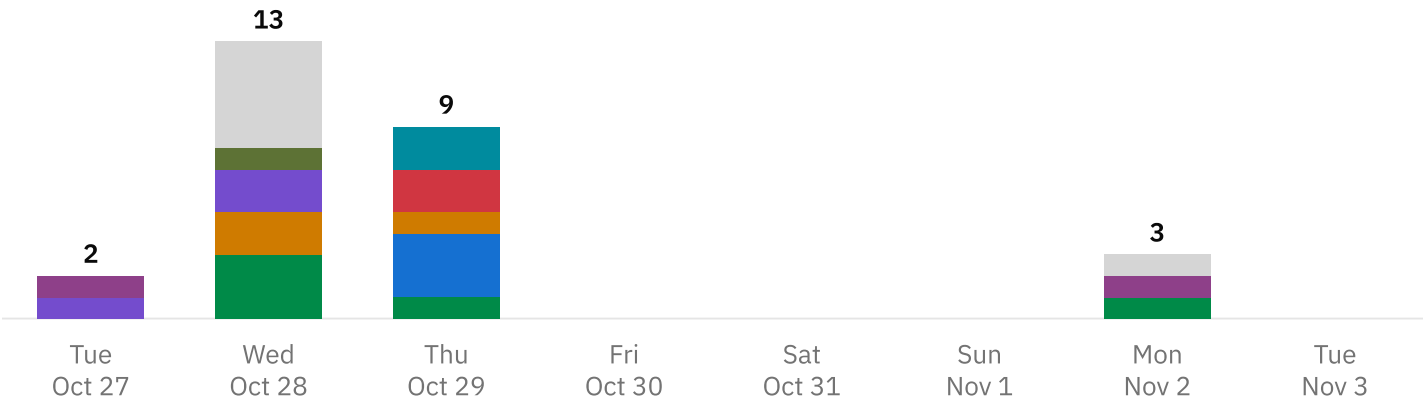
Scrolling an attached Codex or Claude session: 16.7 ms to paint before, 0.15 ms now Median wheel-to-paint is 0.15 ms at 120×30 and 0.21 ms at 240×64, against 0.03 to 0.06 ms for a direct PTY.

Unicode output: mixed graphemes processed 6× faster, CJK text 5× faster Mixed graphemes run at 129 MB/s and CJK at 118 MB/s, allocating about 1.25 MB and 1.48 MB per MiB.

Allocations per read of Claude output: 6,153 before, 119 now, with 33% fewer bytes Retained lines share an append-only text arena, and idle terminals free unused emulator storage when their checkpoint is due.

When it shipped

27 PRs over 4 days, colored by workstream.



Tuesday
Exact checkpoints and idle compaction.

Thursday
Persistent groups, live previews, the eight-pane grid and faster scrolling.

Wednesday
Session overview, Unicode speedup, Codex and Claude import, home-row keys.

Monday
The line arena and the dock audit.

New features

Persistent terminal groups replace the flat picker, and the dock lists every working key

5 PRs, merged Oct 28 to Nov 2

Before

ptyd sessions were a flat list. Seeing which needed attention meant fetching each one, and once finished work piled up the picker was hard to scan.

Now you can

Open a named group of terminals with `ptyd new`, `ptyd ls` or `ptyd attach`, preview and take over any of them, and press quick `jk` to go back from any level.

How it works

ptyd already kept terminals alive across disconnects. What was missing was a way to see them as one piece of work. A group is a saved record that holds several terminals, so a person or an agent can come back to it later, and a keyboard claim decides whose input reaches a terminal when both are there.



From the README demo: a group's terminals on the left and a read-only live preview of the selected one.

What it does

- `ptyd status --json` and `GET /overview` return compact summaries, and default full-history waits allocate 66% fewer bytes in the retained-output fixture #6
- Home-row keys: `i` enters a session and quick `jk` returns, while Escape, control and function keys pass through to the app #13
- Agents keep their terminal ids and HTTP calls, and new `/groups` routes let an agent put its workers in the same group #16
- A person's keyboard claim rejects competing automation input, and read-only observation keeps working #16
- A test presses every dock control, and the dock wraps at 40×16, 80×24 and 120×30 instead of dropping keys #27

Show it

1. Run `ptyd new website` and launch a few terminals in it.
2. Type quick `jk` to leave, then run `ptyd ls` to see the group with a live preview.
3. Press Enter to open the group; the dock at the bottom lists every key that works for the selection.

The change, in code

The group API an agent sees: groups are a new resource, and terminals keep their existing ids. #16

internal/api/groups.go +9

9 unmodified lines

```
func (s *Server) registerGroupRoutes(mux *http.ServeMux) {
  mux.HandleFunc("GET /groups", s.handleGroups)
  mux.HandleFunc("POST /groups", s.handleCreateGroup)
  mux.HandleFunc("GET /groups/{group}", s.handleGroup)
  mux.HandleFunc("PATCH /groups/{group}", s.handleUpdateGroup)
  mux.HandleFunc("DELETE /groups/{group}", s.handleRemoveGroup)
  mux.HandleFunc("PATCH /terminals/{id}", s.handleUpdateTerminal)
  mux.HandleFunc("POST /terminals/{id}/relaunch", s.handleRelaunch)
}
```

The idea that made it work

Make the dock the documentation. #27 removed the `:` palette, `?` help and three key aliases, and every key that works for the current view is labelled with what it will do, such as `a archive` or `a unarchive`.

Engineering details–

Session overview. `ptyd status --watch` opened a live selector with output cursors and match counts. Reclaiming oversized buffers after quiet bursts cut the tested ring's retained heap by about 20% without dropping output. #6

Home-row controls. The Ctrl-backslash shortcut is gone. Quick literal `jk` is reserved, so pause between the letters or use bracketed paste; key repeats and modified keys never trigger it. #13

Live work first. The default picker shows live work, groups terminals and agents, and keeps completed screens under `h` without restarting them. New PTYs use the viewer's display settings instead of the daemon's. #14

Groups. Existing records enter Inbox without restarting, and single-terminal commands moved under `ptyd term`. Rename, move, archive and close-with-confirmation work on groups. #16

Dock audit. Pressing every advertised key on a real daemon showed `h` and `a` each meant two unlabelled things, and the dock never showed `r`, `m`, `x`, `a` or `h`. Labels now name the action for the current selection. #27

Up to 8 live terminal previews in one grid, searchable by screen text and scrollback

3 PRs, merged Oct 29

Before

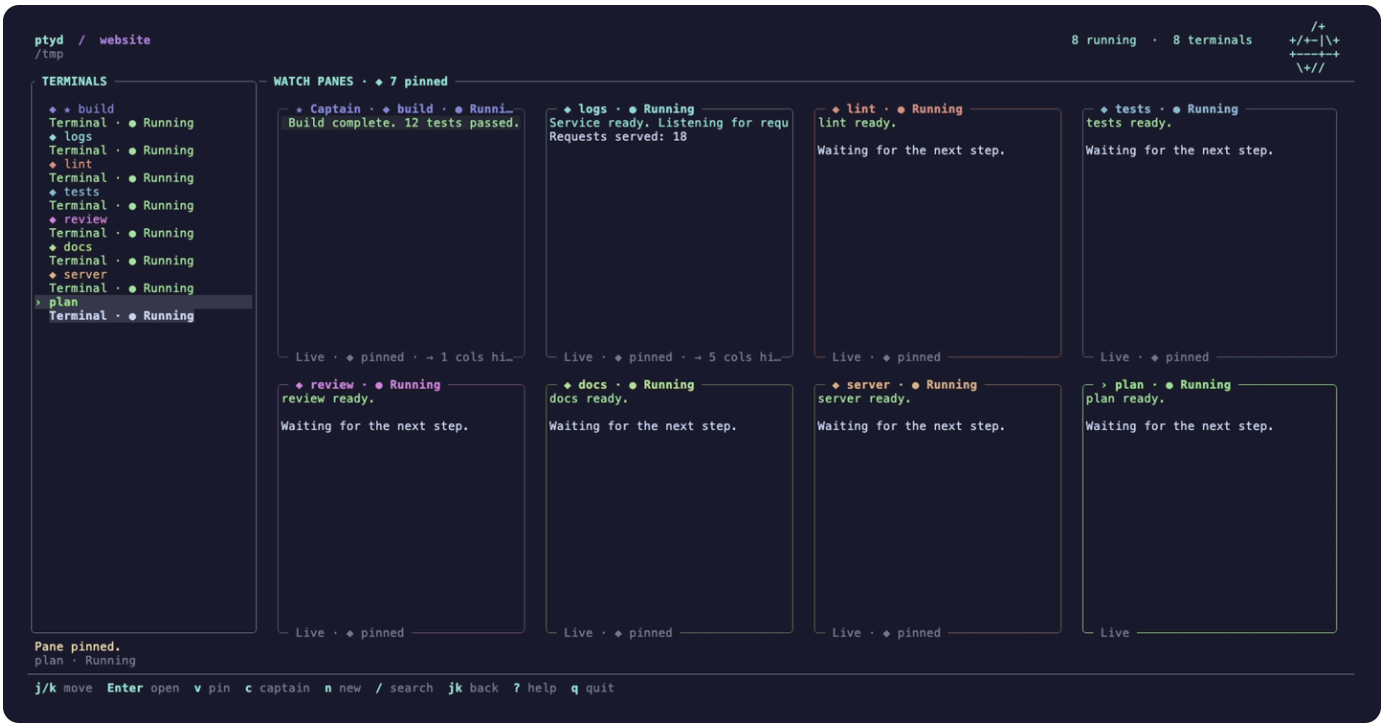
Checking on several agents meant opening each terminal in turn, and `ptyd ls` printed a table that looked empty.

Now you can

Watch up to seven pinned panes plus the selected terminal, and press `/` to find a session by what is on its screen or in its scrollback.

How it works

A preview is a read-only copy of a terminal's current screen. ptyd reads one atomic snapshot per terminal and paints only the cells inside its panel, so watching never resizes the program, counts as a viewer or takes the keyboard.



From the README demo: seven pinned panes and the selected terminal in one group.

What it does

- Eight-pane frames allocate about 0.60 MB of temporary memory, down from 1.56 MB, with the same output bytes #23
- Pins went from 3 to 7 plus the selected terminal, and they persist across browser restarts #18, #23
- Search reads screens and retained scrollback, and labels combine Codex or Claude with the conversation title #23
- A mouse report split right after Escape no longer interrupts Codex or lands in its draft #23
- Previews keep colors and Unicode, while `ls --plain`, JSON, pipes and dumb terminals keep plain output #17

Show it

1. Run `ptyd ls` and move between groups to see each one's live preview.
2. Open a group and press `v` on up to seven terminals to pin them.
3. Press `/` and type text from any screen to jump to that terminal.

The idea that made it work

Previews use the daemon's existing reads, so the browser and grid are a client-only update that leaves running terminals alone. The screen route gained a binary snapshot format so custom palettes and Unicode modes reach the preview intact.

Engineering details–

Group browser. Moving between groups shows their terminals and a colored preview before entering; smaller windows stack the preview below the list. Empty workspaces explain how to start and where saved Inbox terminals are. #17

Back and pins. Quick `jk` goes back through menus and prompts too, and late responses cannot take the keyboard after you leave. `:motion` and `PTYD_REDUCED_MOTION=1` turn off the rotating header. #18

Grid, mouse and captain. Rows, panes and the bottom controls take mouse clicks. Preview scratch storage and HTTP connections are reused, search matches are indexed, and hidden polling pauses during attachment. #23

Codex and Claude sessions from other tabs open in ptyd; a live check finds 5 of 5

3 PRs, merged Oct 28 to Oct 29

Before

Codex and Claude conversations started in another terminal tab were invisible to ptyd, so an agent's work lived in two places.

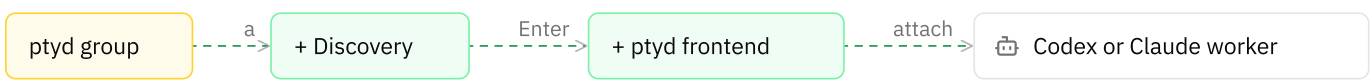
Now you can

Pick a running Codex or Claude conversation in ptyd and press Enter. Detaching returns to the list while the native worker keeps running.

How it works

Each agent CLI keeps its own record of which conversations are running. ptyd asks that record for an exact conversation id, then checks the worker's process identity before and after it opens a frontend, so a conversation that changed in between is declined instead of attached to the wrong worker.

Before | After



How a conversation from another tab reaches a ptyd frontend

What it does

- A live check found 3 Codex and 2 Claude tabs missing from native listings; the read-only probe now finds all 5 #15
- Import rechecks the conversation id, worker pid and process identity after the frontend starts, and closes the frontend if any changed #11
- Codex keeps its composer background: ptyd captures the viewer's colors and answers startup color queries before anyone attaches #15
- The launch menu starts Codex and Claude with full access, `f` switches to CLI settings, and `R` relaunches a finished agent #20

Show it

1. Start Codex in another tab, or run `/bg` in a foreground Claude session.
2. Run `ptyd ls`, open a group and press `a` to list outside agents.
3. Select the conversation and press Enter, then type quick `jk` to return; the worker keeps running.
4. Exit the agent and press `R` to relaunch it with its original flags.

The change, in code

After the frontend starts, the native record is read again; any change in worker, endpoint or process identity closes the frontend. #11

internal/terminal/import.go +16

112 unmodified lines

```
113   if err == nil && (current.Ref != observed.Ref || current.CWD != observed.CWD || (c
114     err = fmt.Errorf("native worker changed during import; refresh and try again")
115   }
116   if err == nil && current.WorkerPID > 0 {
117     after, e := procIdentity(current.WorkerPID)
118     if e != nil || (identity != "" && after != identity) {
119       err = fmt.Errorf("native process identity changed during import")
120     }
121   }
122   if err == nil {
123     err = ctx.Err()
124   }
125   if err != nil {
126     _ = m.Close(info.ID)
127     return ImportResult{}, fmt.Errorf("%w: %v", ErrImportDeclined, err)
128   }
```

The idea that made it work

The agent's own runtime is the only authority. ptyd never guesses a conversation from a window title or process name. An original tab with no exact id is listed with guidance but cannot be imported, and an incomplete or ambiguous listing declines the import.

Engineering details–

Discovery and import. Codex must use its shared local daemon, foreground Claude sessions need `/bg` in their original tab, and stopped Claude conversations need explicit `--resume`. Temporary frontends expire if unclaimed and close on the last live detach. #11

Session controls and original tabs. Attached views reserve a bottom row for the session name and the `jk` hint, drawn from ptyd's terminal model. A frontend that fails keeps its final error screen until `jk`, including a crash during synchronized output. #15

Launch and relaunch. Built-in choices carry their own launch flags, custom commands keep their exact shell arguments, and restored agents keep their original flags. Input queued for the old program cannot turn into menu commands. #20

Fixes

Restarted terminals now come back exact, with 7 kinds of lost state checked byte for byte

3 PRs, merged Oct 27 to Oct 28

What people saw

After ptyd restarted, your terminals came back subtly wrong: saved cursor positions gone, a full-screen app's second screen gone, and keys caught mid-sequence lost.

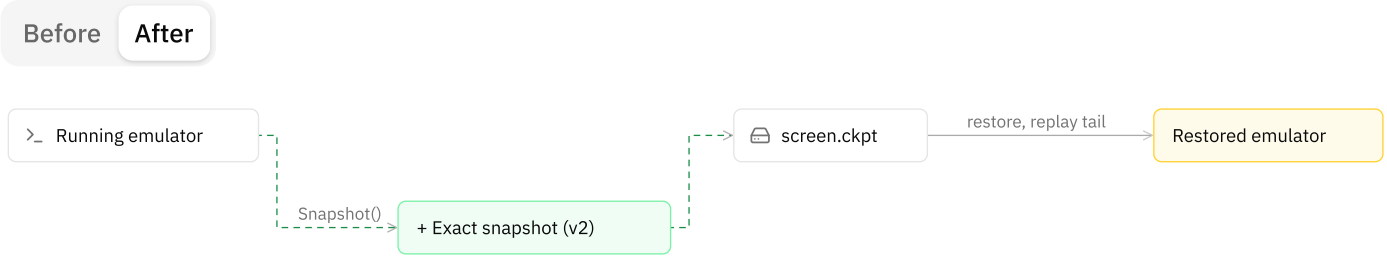
Every version 1 checkpoint was a repaint of up to about 300 KiB, so every restore went through it. A malformed saved screen could also panic ptyd on a later resize (#9).

Now

Checkpoints store the emulator's exact snapshot, old version 1 files still load, and malformed history is rejected before it replaces the terminal.

Why it happened

A terminal emulator holds more than the visible characters. It also has a saved cursor position, a second screen that full-screen apps switch to, and the unfinished tail of an escape sequence still arriving. Repainting the screen as ANSI codes and replaying them keeps only what is visible, so a restored session differed from the one that had been running.



What a checkpoint holds, before and after #2

What changed

- 7 restore cases now match the original byte for byte after checkpoint, tail replay and resize, from REP and the inactive screen to partial UTF-8, CSI and OSC #2
- Version 1 ANSI checkpoints still load, and stay unchanged until the next normal checkpoint replaces them #2
- Prompt markers, colors, cursor saves and independent screen modes survive restore with phosphor v0.4.0 #3
- A saved screen that claims history it never kept is rejected at load instead of panicking during a later resize #9

The change, in code

The checkpoint asks the emulator for its own state instead of a repaint of the screen. #2

internal/terminal/checkpoint.go -1 +1

290 unmodified lines

```
291       return
292     }
293     cols, rows := t.screen.Size()
294 -   body := t.screen.Repaint(true)
294 +   body := t.screen.Snapshot()
295     // Start a new tail generation; output from here on belongs to it.
296     old := p.tail
297     p.gen++
```

Version 2 files restore through phosphor's snapshot reader with the daemon's own size limits; version 1 files still replay as ANSI. #2

internal/terminal/checkpoint.go -2 +16

396 unmodified lines

```
391   scr := phosphor.New(max(cols, 1), max(rows, 1))
392   scr.Write(body)
397   var scr *phosphor.Terminal
398   if hdr.Version == ckptVersion {
399     scr = new(phosphor.Terminal)
400     // These are owner-only files written by this daemon. Preserve the
401     // existing uint16 PTY geometry and storage contract rather than
402     // imposing the library's smaller defaults on accepted sessions.
403     limits := phosphor.SnapshotLimits{
404       MaxBytes: int(^uint(0) >> 1),
405       MaxCells: int(min(uint64(^uint(0)>>1), uint64(65535)*65535)),
406     }
407     if err := scr.RestoreSnapshot(body, limits); err != nil {
408       return nil, 0, fmt.Errorf("corrupt checkpoint in %s: %w", dir, err)
409     }
410     if c, r := scr.Size(); c != cols || r != rows {
411       return nil, 0, fmt.Errorf("checkpoint dimensions differ in %s", dir)
412     }
```

The idea that made it work

Ask the emulator for its state instead of describing the screen to it. phosphor v0.3.0 added a binary snapshot API, so ptyd writes `Snapshot()`, reads it back with `RestoreSnapshot`, and replays the output tail as before.

Engineering details–

Checkpoint format. New checkpoints are version 2 and need ptyd v0.3.0 or later to read. The reader rejects a header and body that disagree, and keeps the daemon's uint16 geometry limits rather than the library's smaller restore defaults. #2

Prompt and editor state. phosphor v0.4.0 keeps colors, prompt markers, cursor saves and independent screen modes in the snapshot, and idle shells allocate smaller emulator rows. The outer header stays at version 2, and tests read a checkpoint written by the released library. #3

Malformed saved history. The fix lives in phosphor's reader, which now validates the structure before a snapshot can replace the terminal. ptyd adopts it with a one-line module bump; the core change carries the exact fuzz regression. #9

Terminal previews: 2,858 cells of background spill before the fix, zero after

2 PRs, merged Oct 29

What people saw

Previews of Codex and Claude cut off their footers and the bottom of the input box, and pane colors bled into the gaps between panes.

A read-only check of live snapshots found 2,858 cells painted outside their panes before #24.

Now

Previews keep the last painted row in view, and every cell outside a pane border matches the frame drawn with no previews, for one through eight panes.

Why it happened

A preview copies part of a larger terminal into a small box. The old code anchored that box to the cursor, but full-screen apps draw footers below it. The row renderer also fills to the right edge of the physical screen with the current background color, which is right for a full terminal and wrong for a cropped pane.

What changed

- 2,858 escaped cells in live snapshots before, 0 after #24
- A new test compares every cell outside pane borders for one through eight panes, with color on and off, and failed on the old renderer #24
- Previews include the last painted row, colored blank cells included, and say how many columns lie outside the pane #19
- Cleanup adds a constant number of control bytes per row and keeps the existing preview storage reuse #24

The change, in code

The preview anchors to the last row with anything painted on it, including a colored blank, instead of the cursor. #19

cmd/ptyd/workspace_preview.go +12

55 unmodified lines

```
56    last := view.Cursor().Y
57  scan:
58    for y := rows - 1; y > last; y-- {
59      for x := 0; x < cols; x++ {
60        c := view.Cell(x, y)
61        if workspacePaintedCell(c) {
62          last = y
63          break scan
64        }
65      }
66    }
67    return min(max(last-max(height, 1)+1, 0), max(rows-max(height, 1), 0))
```

After each cropped row, reset the style and erase to the end of the line so the next pane starts clean. #24

cmd/ptyd/workspace_preview.go +5

42 unmodified lines

```
43      }
44      fmt.Fprintf(out, "\x1b[%d;%dH", r.y+y+1, r.x+1)
45      out.Write(data)
46      // Row inserts fill the host's physical right edge with the active
47      // background. Clear that unpainted tail before later tiles and chrome.
48      if right := r.x + min(r.w, hostCols-r.x); right < hostCols {
49        fmt.Fprintf(out, "\x1b[0m\x1b[%dG\x1b[K", right+1)
50      }
51    }
```

The idea that made it work

Treat a preview as a crop with hard edges. The start row comes from what was painted, and every row ends with a reset and an erase, so later panes and the browser chrome paint over a clean background in their existing order.

Engineering details–

Footers and input boxes. Sparse shells still stay near the top of the pane. Observation leaves the source process and its size alone. The same PR added rounded pane frames, colors derived from the viewing terminal and a shortcut bar with room below it. #19

Background spill. Rows that end at the physical edge keep their final cell. The live check read snapshots without attaching to or changing those terminals. #24

Improvements

Scrolling an attached Codex or Claude session: 16.7 ms to paint before, 0.15 ms now

2 PRs, merged Oct 29

Before

Scrolling Codex or Claude through ptyd lagged a frame behind a direct terminal on every wheel step.

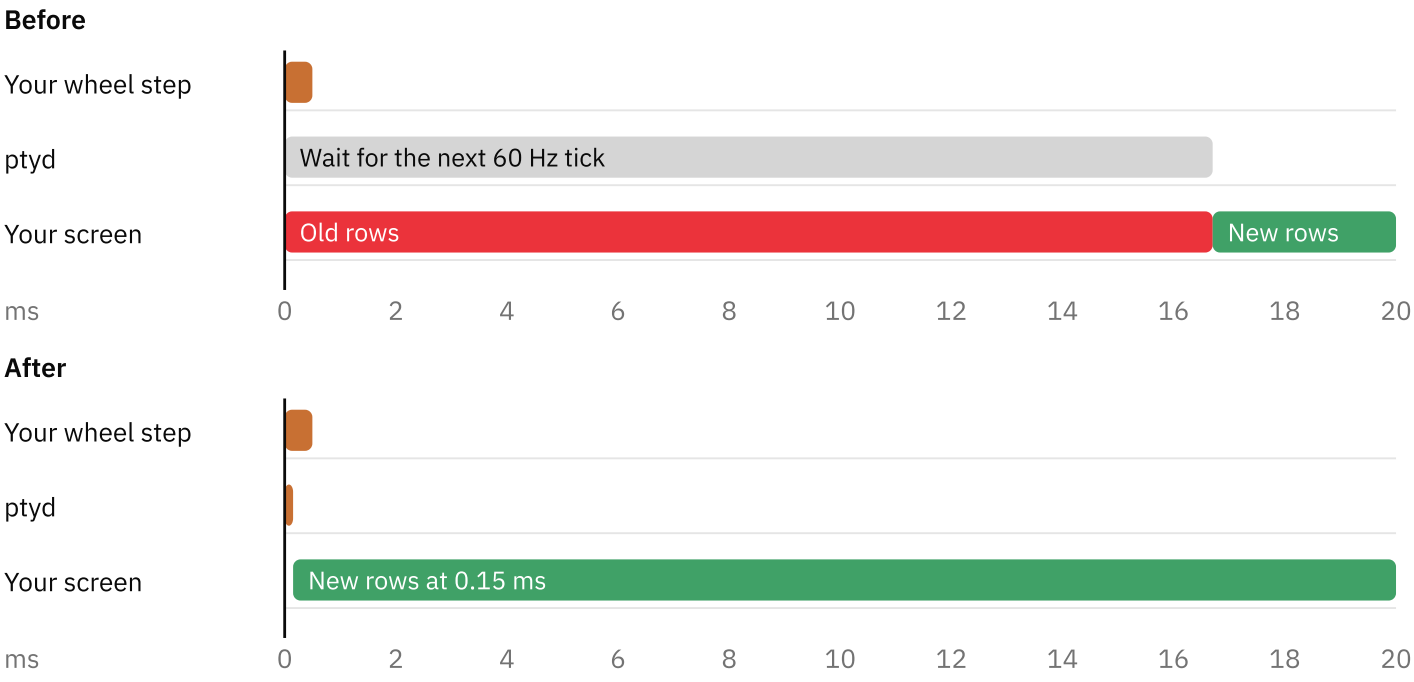
Each wheel step waited a median 16.7 ms for the next 60 Hz paint tick, and every frame resent 6,162 bytes at 120×30.

Now

Median wheel-to-paint is 0.15 ms at 120×30 and 0.21 ms at 240×64, against 0.03 to 0.06 ms for a direct PTY.

Why it happened

The attached view is a terminal drawn inside your terminal. It repainted on a fixed clock of 60 times a second, so input waited for the next tick, and it resent rows the app had redrawn without changing them. The daemon also batched output for throughput, which held back the redraw that a keypress caused.



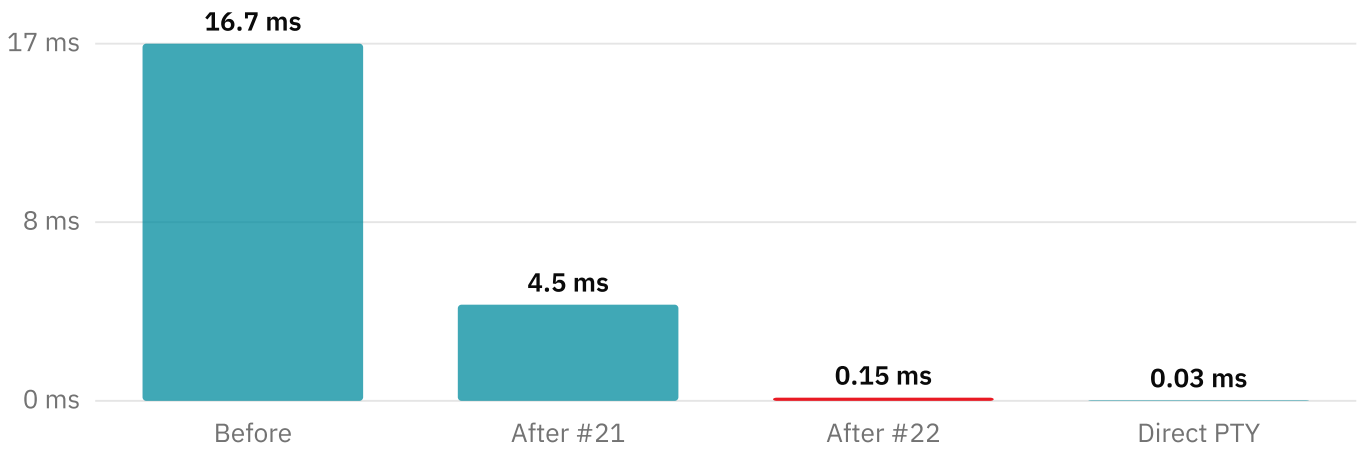
One wheel step at 120×30, median wheel-to-paint from the PR benchmarks.

What changed

- 16.7 ms to 4.5 ms median by painting input responses at once and batching background output at 120 Hz #21

- 4.5 ms to 0.15 to 0.21 ms once the daemon gives output after input a short priority window #22
- Bytes per frame: 6,162 to 385 at 120×30, and 21,242 to 729 at 240×64 #21
- Snapshot setup for 2,000 history rows on a loaded screen: 139 µs to 22.1 µs, with 76% fewer bytes #22

Wheel input to completed paint, median, 120×30



Medians of six alternating local A/B rounds on an Apple M5 Pro, with real PTYs, TCP and a host emulator. No GUI, GPU or display is measured.

The change, in code

The first output after idle paints at once; only background output waits for the 120 Hz budget. #21

internal/client/attach.go +15

246 unmodified lines

```
247     case <-frameReady:
248       // Paint the first output after idle immediately. While busy, merge
249       // background output at 120 paints/second without polling an idle pane.
250       if wait := time.Second/120 - time.Since(lastDraw); wait > 0 {
251         if frameTicks == nil {
252           frameTimer.Reset(wait)
253           frameTicks = frameTimer.C
254         }
255       } else {
256         frameTimer.Stop()
257         frameTicks = nil
258         if err := drawFrame(); err != nil {
259           return result, err
260         }
261       }
```

In the daemon, any input opens a short window in which its output skips bulk coalescing. #22

internal/terminal/attach.go +5

348 unmodified lines

```
349    if !t.running() {
350      return ErrNotRunning
351    }
352    // Large redraws caused by typing or a wheel event are interactive too.
353    // Wake an already-held read as well as bypassing the next coalescing wait.
354    if len(p) > 0 {
355      at.a.prioritizeInput(time.Now().Add(Coalesce))
356    }
357    _, err := t.ptmx.Write(p)
358    return err
```

The idea that made it work

Paint on demand instead of on a clock. Rows are compared as exact rendered bytes, so when the app scrolls, ptyd tells the host terminal to scroll and paints only the newly exposed rows.

Engineering details–

Client renderer. Idle panes have no paint timer. Row comparisons cover both scroll directions, styled blanks, links, Unicode, palettes, screen switches, restore, resize and the status row. Parse and render time per frame fell 32 to 38%. #21

Daemon priority and rollout. A client-only install gives running sessions the new renderer; the daemon change takes effect at its next safe restart. Bulk fanout showed no measured throughput change, and response chunking can add a few allocations per input. #22

Unicode output: mixed graphemes processed 6× faster, CJK text 5× faster

1 PR, merged Oct 28

Before

Output heavy with emoji or Chinese, Japanese and Korean text moved through ptyd at 21 to 23 MB/s, against 366 to 408 MB/s for plain logs.

The emulator allocated about 154 MB of memory for every MiB of mixed Unicode output it read.

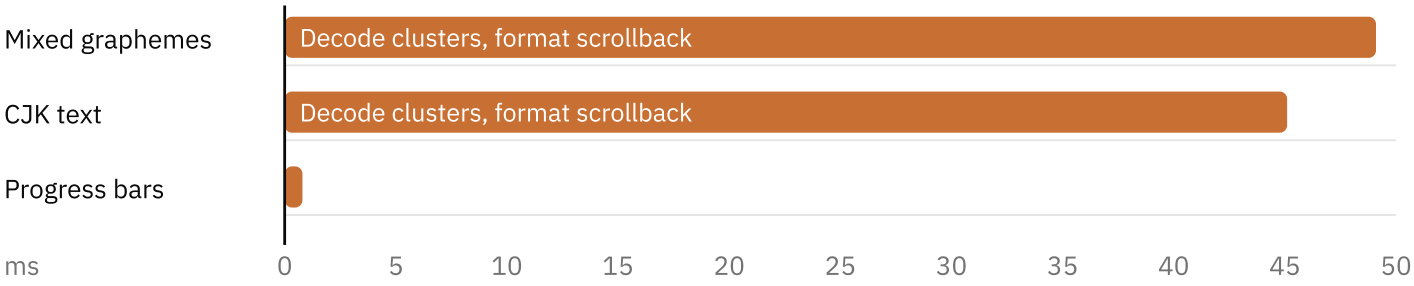
Now

Mixed graphemes run at 129 MB/s and CJK at 118 MB/s, allocating about 1.25 MB and 1.48 MB per MiB.

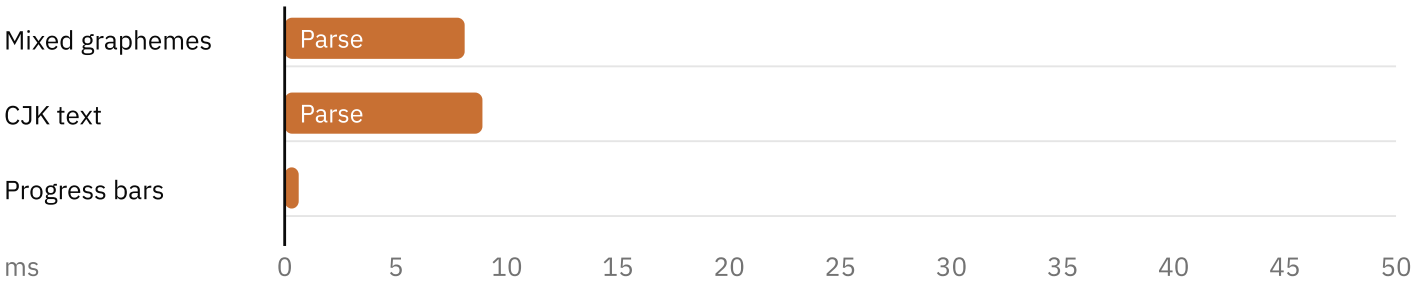
Why it happened

A grapheme is what a reader sees as one character, such as a flag or a family emoji, even when it is made of several code points. Finding where each one ends is expensive, and the old emulator repeated that work while formatting scrollback.

Before



After



Median time for one pass of each fixture through `consume` on one Go thread at 120×40. PTY reads, HTTP and client delivery are excluded.

What changed

- Mixed graphemes: 21.36 to 129.24 MB/s (6.05×) #7
- CJK: 23.27 to 118.01 MB/s (5.07×) #7
- Progress output: 1,311 to 1,659 MB/s (1.27×) #7
- Allocation per MiB of mixed Unicode output: 154 MB to 1.25 MB #7

The idea that made it work

The speedup lives in phosphor, the emulator library. ptyd adopts the published release and refreshes its build profile from macOS and Linux captures, and the HTTP API and saved terminal format stay the same.

Engineering details–

Method. Five alternating 200 ms samples per build ran identical ptyd code against old and new phosphor source. Claude transcripts and logs moved 1.08× and 1.03×, which needs longer measurements to call. #7

Input	Before MB/s	After MB/s
graphemes	21.36	129.24
cjk	23.27	118.01
progress	1,311.12	1,659.20
claude	366.21	394.29
logs	407.95	418.71
claude-persist	268.81	293.62



Allocations per read of Claude output: 6,153 before, 119 now, with 33% fewer bytes

2 PRs, merged Oct 27 to Nov 2

Before

Every read of Claude output made ptyd create a new string for each line, leaving thousands of small objects for Go's garbage collector.

Reading the Claude fixture made 6,153 allocations and 1,056.6 KiB of garbage per read, and about 95% of those bytes came from the line store.

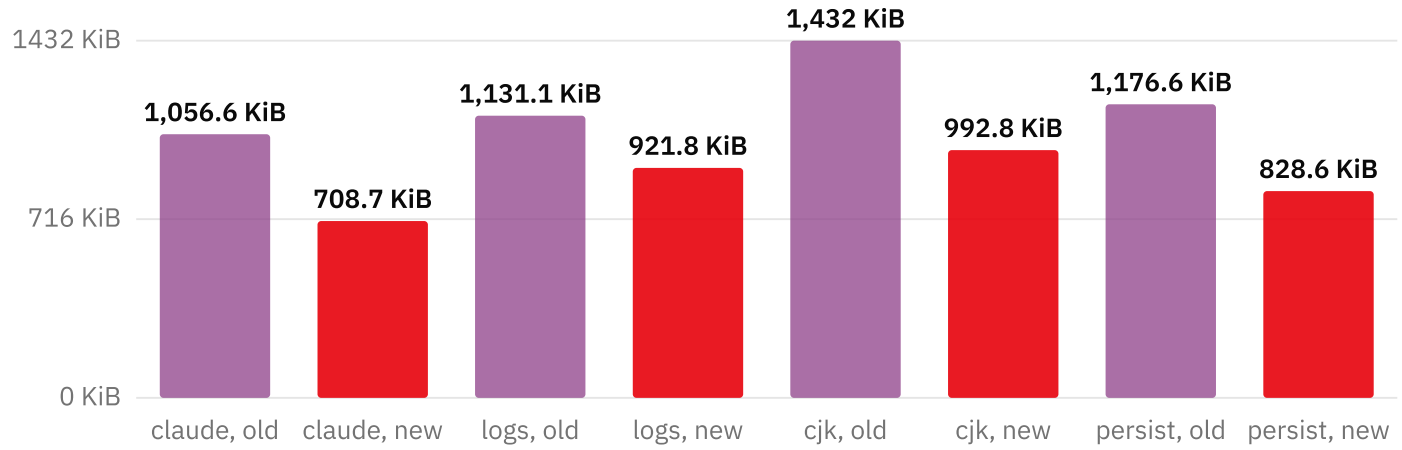
Now

Retained lines share an append-only text arena, and idle terminals free unused emulator storage when their checkpoint is due.

Why it happened

ptyd keeps recent output lines so agents can fetch new output and search it. Allocating each line on its own means thousands of small objects per read, and each one is work for the garbage collector. An arena is one larger buffer that many lines are written into.

Allocated KiB per benchmark op



20 paired, alternating runs per row with `bench/ab.sh` (p=0.000). Whole-`consume` wall time stays within noise because the emulator dominates it.

What changed

- Allocations per op fall 97.8 to 99.7% across the ring and consume benchmarks; Consume/claude goes from 6,153 to 119 #25
- Allocated bytes fall 18 to 33%, from 1,056.6 KiB to 708.7 KiB on Consume/claude #25
- RingWrite is 14% faster, 752.6 µs to 647.9 µs #25
- Idle terminals free erased rows and unused hyperlink and combining-character storage on the existing checkpoint timer #1

The change, in code

Short lines are appended to the arena and returned as strings that alias it; long lines keep their own allocation. #25

internal/terminal/ring.go +15

311 unmodified lines

```
312   size := min(max(minTextArena, 1<<(bits.Len(uint(r.bytes/16)|1)-1)), maxTextArena)
313   if len(text) > size/4 {
314     // A long line gets its own allocation so it never strands a
315     // large unused arena tail.
316     return stripANSIBytes(text)
317   }
318   if cap(r.text)-len(r.text) < len(text) {
319     r.text = make([]byte, 0, size)
320   }
321   start := len(r.text)
322   r.text = appendStripANSI(r.text, text)
323   if len(r.text) == start {
324     return ""
325   }
326   return unsafe.String(&r.text[start], len(r.text)-start)
```

Compaction rides the idle checkpoint, after rechecking under the screen lock that output has stayed quiet. #1

internal/terminal/checkpoint.go +9

256 unmodified lines

```
257   // Check again while holding the screen: output may have resumed since
258   // the timer's first check. Compact before checkpoint I/O can block.
259   t.scrMu.Lock()
260   t.mu.Lock()
261   if !p.stopped && time.Since(t.lastOutputAt) >= ckptSettle {
262     t.screen.Compact()
263   }
264   t.mu.Unlock()
265   t.scrMu.Unlock()
266   t.checkpoint()
```

The idea that made it work

Write stripped line text once into a shared buffer and let each line point into it. Bytes below the write position never change, a full arena is replaced rather than reused, and arena sizes are powers of two so they fit the allocator's size classes.

What we tried first

Arbitrary arena sizes raised retained heap 31%. Seq-free slots cut retained heap 5.6% but made `Ring.read` 3× slower and search 12% slower, and a flat attacher slice showed no change. All three were reverted.

Engineering details–

Line arena. Attention events clone their text because they outlive ring lines and would otherwise pin an arena. Line caps, byte budgets, snapshots and cursors behave as before, and HTTP readers showed no significant change. Retained heap per typical ring went from 752 KiB to 720 KiB. #25

Idle compaction. Compaction adds no timer and no per-output work, and keeps screen cells, retained history and unfinished parser input. The quiet check runs again before compaction and before checkpoint I/O. #1

Everything else

Docs and test-only work: #4, #5 and #10 rewrote the README and guides around what an agent does with ptyd, #8 moved 80 historical benchmark files into a checksummed release bundle and made failed samples stop a comparison, #12 tightened the Codex import witness, and #26 picked up phosphor v0.4.5. (6 PRs)